

ЛАБОРАТОРНА РОБОТА № 7

7.1 Тема

Основи обробки винятків у мові програмування Python

7.2 Мета роботи

Ознайомитися з механізмами обробки виняткових ситуацій у мові програмування Python. Набути практичних навичок використання конструкцій ***try-except***, ***else***, ***finally***, а також створення та застосування користувацьких винятків для забезпечення надійності та коректності програм.

7.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- призначення та роль механізму обробки винятків у мові програмування Python;
- класи стандартних винятків Python (***ZeroDivisionError***, ***ValueError***, ***TypeError***, ***FileNotFoundError***, ***KeyError*** тощо);
- структуру та призначення блоків ***try***, ***except***, ***else***, ***finally***;
- принципи створення та використання користувацьких винятків;
- типові причини виникнення помилок під час виконання програм.

2) уміти:

- використовувати конструкції ***try-except*** для обробки виняткових ситуацій;
- обробляти помилки введення даних користувачем;
- реалізовувати захист від арифметичних помилок (зокрема ділення на нуль);
- здійснювати обробку винятків під час роботи з файлами;
- аналізувати повідомлення про помилки та коректно реагувати на них у програмі.

3) набути навичок:

- написання надійних програм з контролем помилок виконання;

Основи розробки програмного забезпечення мовою програмування Python.

к.т.н., доц. Барбарук Віктор Миколайович

- налагодження програм шляхом аналізу виняткових ситуацій;
- підвищення стійкості програмного коду до некоректних даних;
- застосування винятків для забезпечення логічної завершеності програм;
- документування та фіксації помилок виконання програм.

7.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

7.5 Теоретичні відомості

Поняття помилки та винятку в програмуванні

Під час виконання програм можуть виникати різноманітні помилки, пов'язані з:

- некоректним введенням даних користувачем;
- помилками в арифметичних обчисленнях;
- відсутністю файлів або неможливістю доступу до них;
- некоректною структурою даних або неправильним типом змінних.

У мові програмування Python такі помилки під час виконання програми називаються винятками (exceptions). Виняток – це спеціальна подія, яка перериває нормальний хід виконання програми та сигналізує про помилкову ситуацію.

Якщо виняток не оброблено, програма завершує роботу аварійно з виведенням повідомлення про помилку. Тому коректна обробка винятків є важливою складовою створення надійних програм.

Механізм обробки винятків у Python

Python надає вбудований механізм обробки винятків, який дозволяє:

- перехоплювати помилки під час виконання програми;
- запобігати аварійному завершенню;
- коректно реагувати на помилкові ситуації.

Основною конструкцією для цього є блок ***try-except***.

Загальна структура обробки винятків має вигляд:

- 1) ***try*** – містить фрагмент коду, в якому може виникнути виняток;
- 2) ***except*** – містить код обробки конкретного винятку;
- 3) ***else*** – виконується, якщо у блоці ***try*** не виникло жодного винятку;
- 4) ***finally*** – виконується завжди, незалежно від наявності або відсутності помилок.

Такий підхід дозволяє чітко відокремити основну логіку програми від логіки обробки помилок.

Стандартні винятки Python

Python має велику кількість вбудованих класів винятків. Найпоширеніші з них:

- `ValueError` – виникає, коли значення має правильний тип, але некоректний вміст (наприклад, неможливо перетворити рядок у число);
- `TypeError` – виникає при виконанні операції з несумісними типами даних;
- `ZeroDivisionError` – виникає при спробі ділення на нуль;
- `FileNotFoundError` – виникає, якщо файл не існує;
- `KeyError` – виникає при зверненні до неіснуючого ключа словника;
- `IndexError` – виникає при зверненні до неіснуючого індексу списку.

Знання типів винятків дозволяє програмісту точно визначати причину помилки та реалізовувати коректну реакцію програми.

Обробка кількох винятків

У програмі може виникати декілька різних виняткових ситуацій. Для цього дозволяється використовувати декілька блоків ***except***, кожен з яких обробляє окремий тип винятку.

Такий підхід:

- підвищує читабельність коду;
- дозволяє формувати різні повідомлення для різних помилок;
- спрощує налагодження програм.

Також допускається обробка кількох винятків в одному блоці, якщо реакція на них однакова.

Блок **else** та його призначення

Блок **else** використовується для виконання коду лише у випадку, якщо виняток не виник. Це дозволяє:

- уникнути зайвих перевірок;
- логічно відокремити коректний сценарій виконання програми від обробки помилок;
- зробити програму більш структурованою.

Використання **else** є рекомендованою практикою при розробці навчальних та прикладних програм.

Блок **finally** та завершення роботи програми

Блок **finally** виконується завжди, незалежно від того:

- виник виняток;
- був він оброблений чи ні;
- чи завершилась програма успішно.

Зазвичай блок **finally** використовується для:

- закриття файлів;
- звільнення ресурсів;
- виведення повідомлень про завершення роботи програми.

Це дозволяє гарантувати коректне завершення програми навіть у разі помилок.

Користувацькі винятки

Окрім стандартних винятків, Python дозволяє створювати власні (користувацькі) винятки. Для цього створюється новий клас, який успадковується від класу *Exception*.

Користувацькі винятки застосовуються, коли:

- необхідно обробляти специфічні для задачі помилки;
- стандартних винятків недостатньо для опису ситуації;
- потрібно підвищити зрозумілість логіки програми.

Цей механізм особливо важливий при розробці великих програм та об'єктно-орієнтованих систем.

Роль обробки винятків у забезпеченні надійності програм

Використання механізму обробки винятків дозволяє:

- підвищити стійкість програм до некоректних даних;
- запобігти аварійному завершенню;
- покращити взаємодію з користувачем;
- спростити процес тестування та налагодження програмного коду.

Саме тому обробка винятків є обов'язковим елементом професійного програмування.

7.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.
- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

7.7 Загальне завдання

Розробити програму, яка виконує дії, згідно індивідуального варіанту. Передбачити можливість виникнення та реалізувати коректну обробку винятків. Інформацію про виникнення винятків необхідно виводити на екран та зберігати у логфайл.

7.8 Індивідуальні завдання

- 1) Реалізувати програму з обробкою винятку ділення на нуль.
- 2) Написати програму, яка перевіряє, чи введене значення є цілим числом, та обробляє помилку некоректного вводу.
- 3) Створити програму для введення списку чисел з обробкою винятку у випадку порожнього списку.
- 4) Реалізувати програму, що перевіряє введене число на від'ємність з обробкою відповідного винятку.
- 5) Написати програму з обробкою помилок під час читання даних з текстового файлу.
- 6) Створити програму для перевірки наявності ключа у словнику з обробкою винятку `KeyError`.
- 7) Реалізувати програму, яка обробляє помилки при перетворенні рядка у число.
- 8) Написати програму для обробки арифметичних помилок із записом інформації про виняток у файл.
- 9) Створити програму, яка зчитує дані з файлу та перевіряє їх відповідність заданому формату.
- 10) Реалізувати програму для перевірки коректності введення дати у форматі `dd-mm-yy` з обробкою винятків.

Нульовий варіант (приклад реалізації)

Розробити програму мовою Python, яка:

- зчитує два числа, введені користувачем;
- виконує операцію ділення;
- обробляє можливі винятки:

- введення некоректних даних;
- ділення на нуль;
- у разі виникнення помилки виводить відповідне повідомлення;
- незалежно від результату виконання повідомляє про завершення роботи програми.

Програмний код

```
# Лабораторна робота №7
# Тема: Основи обробки винятків у Python
# Варіант №0 (демонстраційний)

try:
    # Введення даних користувачем
    a = float(input("Введіть перше число: "))
    b = float(input("Введіть друге число: "))

    # Спроба виконати ділення
    result = a / b

except ValueError:
    # Обробка помилки некоректного введення даних
    print("Помилка: введено некоректні дані. Необхідно вводити числа.")

except ZeroDivisionError:
    # Обробка помилки ділення на нуль
    print("Помилка: ділення на нуль неможливе.")

else:
    # Виконується, якщо винятків не виникло
    print(f"Результат ділення: {result}")

finally:
    # Виконується завжди
    print("Програма завершила виконання.")
```

Пояснення до коду

- **try** – містить код, у якому може виникнути помилка;
- **except ValueError** – перехоплює помилки перетворення введених даних у числа;
- **except ZeroDivisionError** – обробляє спробу ділення на нуль;
- **else** – виконується лише тоді, коли помилок не виникло;
- **finally** – виконується незалежно від результату виконання програми.

Результат виконання

```

PS E:\GoogleDrive\python-basic> & 'c:\Python312\python.exe' 'c:\Users\barba\
.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy
\launcher' '1590' '--' 'E:\GoogleDrive\python-basic\test.py'
Введіть перше число: 15
Введіть друге число: value
Помилка: введено некоректні дані. Необхідно вводити числа.
Програма завершила виконання.
PS E:\GoogleDrive\python-basic>

```

7.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

7.10 Контрольні питання

- 1) Як працює конструкція try-except у Python?
- 2) Яке призначення блоку finally?
- 3) Як створити власний виняток у Python?
- 4) Як визначити тип винятку в блоці except?
- 5) У чому різниця між блоками except та else?

7.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>

2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>